

ENGINEERING STANDARD · VERSION 1.0

Spring AI Tool Implementation Standard

How to design, build, register, and expose a tool in Spring AI so it is unambiguous to an AI model, safe to call, and consumable by any agent — in-process or over MCP.

IN THIS DOCUMENT

Naming conventions and the tool contract
Input and output schema design with Java records
A complete, numbered implementation walkthrough
Registering tools with ChatClient-based agents
Exposing tools to external agents via MCP
Testing patterns and a definition-of-done checklist

Framework Spring AI 2.0.1 on Spring Boot 4.0/4.1

Audience Engineers implementing tools for AI agents

Status Adopted standard — apply to all new tools

Primary sources docs.spring.io/spring-ai, spring.io/projects/spring-ai,
habuma.com/springairecipes

Contents

1 Purpose and Scope

- 1.1 Why tools need a standard
 - 1.2 What "consumable by an AI agent" means
-

2 Prerequisites and Project Setup

- 2.1 Dependencies and the BOM
 - 2.2 The two consumption paths
-

3 Core Concepts

- 3.1 The tool-calling loop
 - 3.2 Key types: ToolCallback, ToolDefinition, ToolContext
-

4 The Tool Contract: Naming Conventions

- 4.1 Tool name syntax
 - 4.2 The verb_noun pattern
 - 4.3 Class, package, and parameter naming
 - 4.4 Writing the description
-

5 Input Schema Design

6 Output Schema Design

7 Step-by-Step: Implementing a Tool

- 7.1 Step 1 – Define the input record
 - 7.2 Step 2 – Define the output record
 - 7.3 Step 3 – Write the tool method
 - 7.4 Step 4 – Handle errors explicitly
 - 7.5 Step 5 – Register the tool class as a bean
 - 7.6 Step 6 – Verify the generated schema
-

8 Registering Tools with an Agent

9 Exposing Tools to External Agents via MCP

10 Error Handling

11 Testing Tools

12 Definition-of-Done Checklist

13 Common Pitfalls and Anti-Patterns

A Appendix: Annotation and API Reference

B Appendix: Complete Reference Implementation

C Appendix: Sources

1 Purpose and Scope

This document is the standard this team uses to implement a **tool** in Spring AI: a Java method an AI model can decide to invoke while generating a response. It specifies naming rules, input/output schema design, registration, testing, and how to make a tool reachable by agents outside the process that defines it. Follow it for every new tool; use it as the review checklist for every pull request that adds one.

1.1 Why tools need a standard

A tool is not a private implementation detail. Its name and description are read by a language model — not compiled, not type-checked — to decide *whether* to call it and *how* to fill in its arguments. A method named `process` with no description compiles perfectly and is useless to a model. A tool that returns a raw `String` the model must re-parse invites the model to hallucinate a field that is not there. Both failures are invisible in code review unless the reviewer is checking against a standard, because nothing in the Java type system flags them.

Spring AI itself is permissive by design — it will happily generate a schema from any method signature. This document narrows that permissiveness to a single house style so that every tool in the system reads the same way to a model, to a teammate, and to whichever agent ends up calling it.

1.2 What “consumable by an AI agent” means

A tool built to this standard is consumable in two distinct senses, and a compliant tool supports both without changing its core logic:

CONSUMPTION PATH	WHAT "CONSUMABLE" REQUIRES
In-process — a <code>ChatClient</code> -driven agent inside your own application calls the tool as part of answering a prompt.	A <code>@Tool</code> -annotated method or a <code>ToolCallback</code> bean, registered on the <code>ChatClient</code> . Covered in §7–8.
External — an agent running in a different process (Claude Desktop, a separate Spring AI service, any MCP client) discovers and calls the tool over the network.	The same method exposed as an MCP server tool via <code>@McpTool</code> , so any MCP client can list it, read its schema, and invoke it. Covered in §9.

THE POINT OF THIS STANDARD

A tool built correctly against §4–6 needs no rework to move from the first path to the second. The naming rule, the input record, and the output record are shared; only the registration step differs. Get the contract right once.

2 Prerequisites and Project Setup

2.1 Dependencies and the BOM

This standard targets **Spring AI 2.0.1** on **Spring Boot 4.0.x–4.1.x**, the current GA line as of this writing. All Spring AI artifacts are on Maven Central; no milestone or snapshot repository is needed for a release build.

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.springframework.ai</groupId>
      <artifactId>spring-ai-bom</artifactId>
      <version>2.0.1</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>

<dependencies>
  <!-- Pulls in spring-ai-model, which supplies @Tool, @ToolParam,
       ToolCallback, and the JSON schema generator used throughout
       this document. Transitive on every spring-ai-starter-model-*. -->
  <dependency>
    <groupId>org.springframework.ai</groupId>
    <artifactId>spring-ai-starter-model-openai</artifactId>
  </dependency>

  <!-- Only if this tool must also be reachable by an external agent
       over MCP. See §9. -->
  <dependency>
    <groupId>org.springframework.ai</groupId>
    <artifactId>spring-ai-starter-mcp-server-webmvc</artifactId>
  </dependency>
</dependencies>
```

Substitute the model starter for your provider — the artifact id follows the pattern `spring-ai-starter-model-{provider}` (`spring-ai-starter-model-anthropic` , `spring-ai-starter-model-vertex-ai-gemini` , and so on). Tool-calling support is identical across providers; Spring AI's `ToolCallingManager` abstracts the wire-format differences.

2.2 The two consumption paths

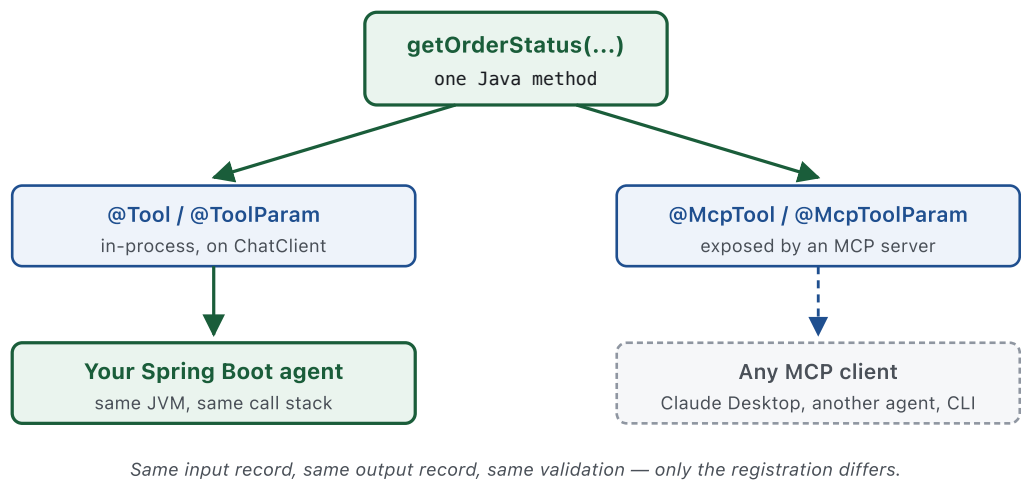


Figure 2.1 — One tool, two front doors. Sections 7 and 8 build the method and register it in-process. Section 9 adds `@McpTool` to the same class so an external agent can reach it — no change to the method body.

3 Core Concepts

3.1 The tool-calling loop

Before writing a tool, it helps to see exactly where it sits in a request. The model never executes your code. It emits a structured request naming the tool and its arguments; Spring AI's `ToolCallingManager` matches that name against the registered tools, invokes the Java method, serializes the return value, and feeds it back to the model as part of the same conversation.

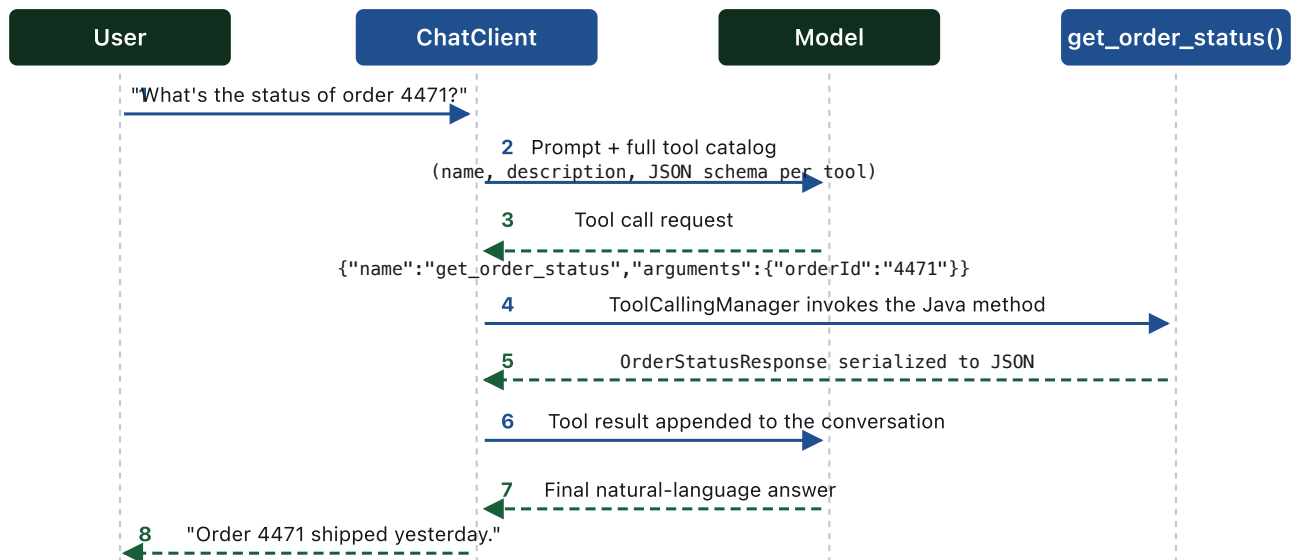


Figure 3.1 — The tool-calling loop. Steps 4–5 are the only part your code controls, and Spring AI can repeat steps 2–6 several times in one turn if the model chains tool calls. Everything downstream of step 3 depends on the `name` and JSON schema built in §4–6 being unambiguous — the model cannot ask a Java compiler for help; it can only re-read what step 2 sent it.

3.2 Key types: ToolCallback, ToolDefinition, ToolContext

Three types from `org.springframework.ai.tool` carry the contract end to end. You will not usually construct them by hand — `@Tool` and the JSON schema generator do it for you — but understanding their shape explains what the annotations in §4–7 actually produce.

```

public interface ToolDefinition {
    String name();           // unique within the tool set given to one model call
    String description();    // the only signal the model gets for WHEN to call this
    String inputSchema();    // JSON Schema, generated from your input record
}

public interface ToolCallback {
    ToolDefinition getToolDefinition();
    ToolMetadata getToolMetadata();           // e.g. returnDirect
    String call(String toolInput);
    String call(String toolInput, ToolContext toolContext);
}

```

`ToolContext` (`org.springframework.ai.tool.ToolContext`) is the escape hatch for data a tool needs but the model must never see or supply — a tenant id, an authenticated user id, a request-scoped trace id. It is populated by your application code when invoking the agent, not by the model, and it is excluded from the generated JSON schema automatically.

4 The Tool Contract: Naming Conventions

This is the section to enforce in code review. A tool's name and description are the *only* information the model has when deciding whether to call it — there is no IDE autocomplete, no compiler, no javadoc lookup on the model's side. Get this section right and the rest of the implementation is mechanical.

4.1 Tool name syntax

Spring AI's own requirement is only that a name be **unique within the tool set given to one model call**. That is not enough to guarantee interoperability: OpenAI, Anthropic, Amazon Bedrock, and the Model Context Protocol itself all constrain tool identifiers to the same pattern —

```
^[a-zA-Z0-9_-]{1,64}$
```

This standard adopts a stricter subset of that cross-provider rule:

RULE	RATIONALE
snake_case , lowercase only	Matches OpenAI's and Anthropic's own function libraries, so a tool reads consistently whether it came from your code or a third party. Mixed case in a name the model must reproduce verbatim invites transcription errors.
Pattern: <code>^[a-z][a-z0-9]*(_[a-z0-9]+)*\$</code>	A stricter subset of the provider-wide <code>^[a-zA-Z0-9_-]{1,64}\$</code> . No hyphens, no leading digit, no double underscores.
1–64 characters, target ≤ 40	64 is the hard ceiling every provider enforces. Beyond ~40 characters a name is usually trying to be a description — move that detail into the <code>description</code> attribute instead (§4.4).
Unique across the whole registered tool set, not just per class	Spring AI enforces per-call uniqueness, but two unrelated <code>OrderTools</code> and <code>BillingTools</code> classes can both produce <code>get_status</code> and silently shadow one another the day they are registered together. Prefix with the domain if collision is plausible: <code>order_get_status</code> , <code>billing_get_status</code> .

4.2 The verb_noun pattern

Every tool name follows one of two shapes:

```
<verb>_<noun>           get_order_status
<domain>_<verb>_<noun>  billing_cancel_invoice  # when domain-prefixing is needed
```

VERB	USE FOR	SIGNALS TO THE MODEL
<code>get</code>	Read a single resource by identifier.	Safe, idempotent, no side effects.
<code>list</code> / <code>search</code>	Read a collection, optionally filtered.	Safe, idempotent, may return zero or many.
<code>create</code>	Create a new resource.	Side-effecting, not idempotent — each call may create a new record.
<code>update</code>	Modify an existing resource.	Side-effecting, typically idempotent on the same input.
<code>cancel</code> / <code>delete</code>	Terminate or remove a resource.	Side-effecting, destructive — pair with the MCP <code>destructiveHint</code> in §9.
<code>calculate</code> / <code>convert</code>	Pure computation, no I/O.	Safe, idempotent, deterministic.

NEVER USE A BARE JAVA METHOD NAME AS THE TOOL NAME

`process`, `handle`, `execute`, `doWork` pass code review as method names and fail this standard as tool names — they tell the model nothing about what happens or when to invoke it. If `@Tool` is left without an explicit `name`, Spring AI defaults to the method name verbatim, so an implementer who forgets the attribute silently ships a non-compliant name. **Always set `name` explicitly**, even when it happens to match the method name.

	NAME	WHY
GOOD	<code>get_order_status</code>	Verb + noun, unambiguous, safe to call speculatively.
GOOD	<code>cancel_subscription</code>	Verb signals a destructive, non-idempotent action; the model can weigh that before calling it.
GOOD	<code>convert_currency</code>	Pure function, no ambiguity about side effects.
BAD	<code>orderStatus</code>	camelCase; no verb, so "get" is implied but "cancel" would look identical.
BAD	<code>Get_Order_Status</code>	Mixed case violates the lowercase rule.
BAD	<code>doOrderStuff</code>	Vague verb, vague noun — tells the model nothing.
BAD	<code>get_order_status_by_order_id_from_the_fullment_service</code>	Implementation detail (which backend) leaked into the name; over length target. That belongs in the description, if anywhere.

4.3 Class, package, and parameter naming

The tool name is snake_case because it crosses the model boundary; everything on the Java side of that boundary stays ordinary Java style. Do not let the tool-name convention leak into your class or field names.

ELEMENT	CONVENTION	EXAMPLE
Tool provider class	<code>{Domain}Tools</code> , standard PascalCase	<code>OrderTools</code> , <code>BillingTools</code>
Package	<code>{basePackage}.tools</code>	<code>com.acme.orders.tools</code>
Java method name	Standard camelCase; may equal the tool name's camelCase form but is not required to	<code>getOrderStatus</code>
Input record	<code>{ToolName}Request</code> , PascalCase from the tool name	<code>GetOrderStatusRequest</code>
Output record	<code>{ToolName}Response</code>	<code>GetOrderStatusResponse</code>
Record fields / parameters	Standard camelCase; Jackson serializes to the same camelCase in the JSON schema by default	<code>orderId</code> , <code>includeHistory</code>

4.4 Writing the description

Spring AI's own documentation calls the description *"strongly recommended—without it the model has no guidance on when to call the tool."* This standard makes it **mandatory** and requires it to answer three questions in order: **what** the tool does, **when** to call it, and **what it needs** to do so.

```
// what it does · when to call it · what it needs
@Tool(
    name = "get_order_status",
    description = "Retrieves the current fulfillment status of one order. "" +
        "Call this when the customer asks about the status, tracking, "" +
        "or delivery date of an order they already placed. Requires "" +
        "the order ID; ask the customer for it if not yet known."
)
```

DESCRIPTION

BAD

"Gets order status" — restates the name; gives the model no help deciding *when*, and none of the disambiguating detail a model needs when several similarly named tools are in the same catalog.

BAD

"Calls the FulfillmentServiceClient.fetchStatus REST endpoint" — describes the implementation, which is irrelevant to the model and will go stale the day the backend changes.

GOOD

"Retrieves the current fulfillment status of one order. Call this when the customer asks about status, tracking, or delivery date."

DISAMBIGUATE NEAR-DUPLICATE TOOLS EXPLICITLY

If your catalog has both `get_order_status` and `get_shipment_status`, state the distinction in both descriptions (*"an order may span several shipments; use this when the customer asks about the whole order rather than one package"*). The model resolves ambiguity by re-reading descriptions, not by asking you.

5 Input Schema Design

A tool method may take individual primitive parameters and Spring AI will happily generate a schema from them. This standard forbids that for anything but the most trivial tool, and requires a dedicated **request record** instead.

RULE

Every tool with one or more parameters declares a single input parameter of a dedicated `record` type named `{ToolName}Request`. Every field is annotated with `@ToolParam`. No tool method takes more than one business parameter plus an optional trailing `ToolContext`.

WHY A RECORD, NOT LOOSE PARAMETERS	CONSEQUENCE OF IGNORING IT
The generated JSON Schema names its properties after record fields, which are self-documenting and stable under refactoring.	Loose parameters are schema'd by position-derived names in some reflection paths and break silently on a parameter reorder.
Validation annotations (<code>@NotNull</code> , <code>@Pattern</code> , Bean Validation) live in exactly one place.	Validation logic gets duplicated inside the method body, and drifts from the schema the model was actually shown.
The record is directly reusable as the MCP tool's input type in §9 — no translation layer.	A loose-parameter tool needs a wrapper record written later anyway, doubling the work.
Unit tests construct one object, not an N-argument call that silently compiles if two <code>String</code> parameters are swapped.	Two adjacent <code>String</code> parameters (<code>fromCurrency</code> , <code>toCurrency</code>) are a classic swap bug the compiler cannot catch.

EXAMPLE

```
public record GetOrderStatusRequest(  
    @ToolParam(description = "The order identifier, as shown on the customer's receipt or confirmation  
email.")  
    String orderId,  
  
    @ToolParam(description = "Include the full status history, not just the current status.", required =  
false)  
    Boolean includeHistory  
) {}
```

FIELD RULE	DETAIL
Every field has a description	Same rationale as §4.4 — the model fills in arguments purely from these strings.
Mark optional fields explicitly	<code>@ToolParam(required = false)</code> . Spring AI's precedence order also honors Jackson's <code>@JsonProperty(required = false)</code> , Swagger's <code>@Schema(required = false)</code> , and <code>@Nullable</code> , in that order — but set <code>@ToolParam</code> explicitly rather than relying on a fallback.
Use boxed types for optional fields	<code>Boolean</code> , not <code>boolean</code> — a primitive cannot represent "the model omitted this."
Never accept <code>Optional<T></code>	Unsupported by Spring AI's method-tool reflection. Use a boxed type plus <code>required = false</code> instead.
No implementation types	No JPA entities, no framework-specific request objects. The record is a wire contract, not a domain object — map to your domain type inside the method body.

Spring AI's `JsonSchemaGenerator` turns the record above into:

```
{
  "type": "object",
  "properties": {
    "orderId": {
      "type": "string",
      "description": "The order identifier, as shown on the customer's receipt or confirmation email."
    },
    "includeHistory": {
      "type": "boolean",
      "description": "Include the full status history, not just the current status."
    }
  },
  "required": ["orderId"]
}
```

6 Output Schema Design

RULE

Every tool returns a dedicated record named {ToolName}Response . No tool returns a bare String , Map<String,Object> , or a domain entity directly.

WHY A RECORD, NOT A RAW TYPE	CONSEQUENCE OF IGNORING IT
Spring AI's default ToolCallResultConverter serializes the return value with Jackson. A record gives the model a stable, named, typed shape to read back.	A bare String forces the model to parse free text out of a return value that was never designed to be parsed — the classic source of a model hallucinating a field.
A response record can carry a structured error alongside a successful payload (see §10), which the model can branch on.	A raw exception message concatenated into a returned string gives the model no reliable way to distinguish "no matching order" from "order shipped, no tracking number yet."
The exact same record becomes the MCP outputSchema in §9 when generateOutputSchema = true .	A Map return type produces an MCP output schema of {"type":"object"} with no properties — useless to an external agent.

EXAMPLE

```
public record GetOrderStatusResponse(  
    String orderId,  
    OrderStatus status,  
    // null when the order has not shipped yet  
    String trackingNumber,  
    // null when includeHistory was false or not requested  
    List<StatusEvent> history  
) {  
    public enum OrderStatus { PENDING, PROCESSING, SHIPPED, DELIVERED, CANCELLED }  
    public record StatusEvent(OrderStatus status, Instant occurredAt) {}  
}
```

RULE	DETAIL
Prefer enums over free-text status strings	<code>OrderStatus</code> above, not a bare <code>String status</code> . An enum constrains what the model can read back to the user and is self-documenting in the generated schema's <code>enum</code> array.
Document nullability with a comment or <code>@Nullable</code>	A model that infers "no tracking number" means "order not found" instead of "not shipped yet" will confidently tell the customer something false.
Keep it flat where possible	Deep nesting is legal but costs the model more tokens and more chances to misread a field path. Two levels is a reasonable ceiling for most tools.
Never return a stack trace or exception <code>toString()</code>	See §10 for the correct error shape.

7 Step-by-Step: Implementing a Tool

This section builds one complete tool, `get_order_status`, from nothing to registered and callable. Every step applies the rules from §4–6; nothing here is new policy, only its application in order.

7.1 Step 1 — Define the input record

Create the request record in the same package as the tool class, per §5.

```
// com/acme/orders/tools/GetOrderStatusRequest.java
package com.acme.orders.tools;

import org.springframework.ai.tool.annotation.ToolParam;

public record GetOrderStatusRequest(

    @ToolParam(description = "The order identifier, as shown on the customer's receipt or confirmation email.")
    String orderId,

    @ToolParam(
        description = "Include the full status history, not just the current status.",
        required = false
    )
    Boolean includeHistory
) {}
```

7.2 Step 2 — Define the output record

```
// com/acme/orders/tools/GetOrderStatusResponse.java
package com.acme.orders.tools;

import java.time.Instant;
import java.util.List;

public record GetOrderStatusResponse(
    String orderId,
    OrderStatus status,
    String trackingNumber,
    List<StatusEvent> history
) {
    public enum OrderStatus { PENDING, PROCESSING, SHIPPED, DELIVERED, CANCELLED }
    public record StatusEvent(OrderStatus status, Instant occurredAt) {}

    // Convenience factory keeps "not found" out of the constructor call sites.
    public static GetOrderStatusResponse notFound(String orderId) {
        return new GetOrderStatusResponse(orderId, null, null, List.of());
    }
}
```

7.3 Step 3 — Write the tool method

Create the tool provider class following the `{Domain}Tools` convention from §4.3. Inject collaborators through the constructor exactly as with any other Spring bean — a tool class is not special.

```
// com/acme/orders/tools/OrderTools.java
package com.acme.orders.tools;

import com.acme.orders.tools.GetOrderStatusResponse.OrderStatus;
import com.acme.orders.tools.GetOrderStatusResponse.StatusEvent;
import org.springframework.ai.tool.annotation.Tool;
import org.springframework.stereotype.Component;

@Component
public class OrderTools {

    private final OrderRepository orderRepository;

    public OrderTools(OrderRepository orderRepository) {
        this.orderRepository = orderRepository;
    }

    @Tool(
        name = "get_order_status",
        description = "Retrieves the current fulfillment status of one order. "" +
            "Call this when the customer asks about the status, tracking, "" +
            "or delivery date of an order they already placed. Requires "" +
            "the order ID; ask the customer for it if not yet known."
    )
    public GetOrderStatusResponse getOrderStatus(GetOrderStatusRequest request) {
        var order = orderRepository.findById(request.orderId())
            .orElse(null);

        if (order == null) {
            return GetOrderStatusResponse.notFound(request.orderId());
        }

        List<StatusEvent> history = Boolean.TRUE.equals(request.includeHistory())
            ? order.statusHistory().stream()
                .map(e -> new StatusEvent(OrderStatus.valueOf(e.status()), e.occurredAt()))
                .toList()
            : List.of();

        return new GetOrderStatusResponse(
            order.id(),
            OrderStatus.valueOf(order.status()),
            order.trackingNumber(), // null until shipped – documented on the record
            history
        );
    }
}
```

WHY THE METHOD BODY RETURNS RATHER THAN THROWS ON "NOT FOUND"

A missing order is an expected, common outcome the model needs to relay to the customer — it is not exceptional. Reserve thrown exceptions for genuinely unexpected failures (§10): a database timeout, a downstream service being unreachable. Modeling "not found" as a normal return value keeps the distinction visible to the model.

7.4 Step 4 — Handle errors explicitly

The method above already handles its one expected failure mode as data. For failures the tool genuinely cannot recover from, throw a `RuntimeException` with a message written for the model to relay, not a stack trace:

```
@Tool(name = "get_order_status", description = "...")
public GetOrderStatusResponse getOrderStatus(GetOrderStatusRequest request) {
    if (request.orderId() == null || request.orderId().isBlank()) {
        // A RuntimeException message is sent back to the model by default (§10) –
        // write it as guidance, not as a log line.
        throw new IllegalArgumentException("An order ID is required to look up status.");
    }
    try {
        return lookUp(request);
    } catch (OrderServiceUnavailableException ex) {
        throw new IllegalStateException(
            "The order lookup service is temporarily unavailable. Ask the customer to try again
            shortly.", ex);
    }
}
```

7.5 Step 5 — Register the tool class as a bean

`@Component` on `OrderTools` (Step 3) is sufficient for the class to be discovered by component scanning and injected wherever it is registered with a `ChatClient` in §8. No further registration is needed for the in-process path. If your project builds a native image, one additional step applies:

GRAALVM / SPRING AOT

A tool class compiled to a native image must either be a Spring bean (satisfied here by `@Component`) or carry `@RegisterReflection(memberCategories = MemberCategory.INVOKE_DECLARED_METHODS)`, because reflective invocation of `@Tool` methods otherwise has no runtime hint to preserve. Being a Spring bean is the simpler path and is satisfied by every tool class in this document.

7.6 Step 6 — Verify the generated schema

Before registering the tool with a live model, confirm the schema Spring AI actually generated matches what §5 specified — this is the fastest way to catch a missing `@ToolParam` or an unintended required field. Resolve the `ToolCallback` for the method and print its definition:

```
MethodToolCallback callback = MethodToolCallback.builder()
    .toolDefinition(ToolDefinitions.builder(
        ReflectionUtils.findMethod(OrderTools.class, "getOrderStatus", GetOrderStatusRequest.class))
        .build())
    .toolMethod(ReflectionUtils.findMethod(OrderTools.class, "getOrderStatus",
GetOrderStatusRequest.class))
    .toolObject(orderTools)
    .build();

System.out.println(callback.getToolDefinition().name());           // get_order_status
System.out.println(callback.getToolDefinition().inputSchema());    // the JSON Schema from §5
```

§11 formalizes this check into an automated test that runs on every build rather than a one-off manual step.

8 Registering Tools with an Agent

A tool built to §7 is inert until a `ChatClient` knows about it. Spring AI offers two registration scopes; pick the narrowest one that fits.

PER-CALL REGISTRATION

Scoped to a single prompt. Use this when a tool is relevant to only one code path.

```
String response = chatClient.prompt()
    .user("What's the status of order 4471?")
    .tools(orderTools) // any @Tool-annotated bean, ToolCallback, or ToolCallbackProvider
    .call()
    .content();
```

DEFAULT REGISTRATION

Scoped to every prompt issued by this client. Use this for a stable agent identity that always has the same core toolset available.

```
@Bean
ChatClient supportAgent(ChatClient.Builder builder, OrderTools orderTools, BillingTools billingTools) {
    return builder
        .defaultSystem("You are a customer support assistant for Acme Orders.")
        .defaultTools(orderTools, billingTools)
        .build();
}
```

PER-CALL TOOLS ADD TO, NOT REPLACE, DEFAULTS

Calling `.tools(...)` on a prompt appends to whatever `.defaultTools(...)` already registered; it does not override the set. Design each tool's name to be unambiguous across the union of every registration scope that might be active in the same conversation, per the uniqueness rule in §4.1.

REGISTERING A FUNCTIONAL TOOL AS A BEAN (NO OWNING CLASS)

When the tool wraps an existing service method you do not want to annotate directly — a third-party client, for instance — expose it as a `ToolCallback` bean instead of retrofitting `@Tool` onto code you do not control:

```
@Bean
ToolCallback convertCurrency(ExchangeRateClient client) {
    return FunctionToolCallback.builder("convert_currency", client::convert)
        .description("Converts an amount from one ISO 4217 currency to another using "" +
            "the current exchange rate. Call this whenever the customer asks "" +
            "for a price in a currency different from the one it is quoted in.")
        .inputType(ConvertCurrencyRequest.class)
        .build();
}
```

This still obeys §4–6 in full: the tool name is `snake_case`, the description follows the what/when/needs structure, and `ConvertCurrencyRequest` is a dedicated input record. Only the mechanism of registration differs from §7.

9 Exposing Tools to External Agents via MCP

Everything so far makes a tool callable by a `ChatClient` running in the same JVM. To make it callable by an agent running **anywhere else** — Claude Desktop, a separate microservice, any Model Context Protocol client — expose it through an MCP server. This section adds that without touching the business logic written in §7.

9.1 Dependencies

```
<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-starter-mcp-server-webmvc</artifactId>
</dependency>
```

Use `spring-ai-starter-mcp-server` for STDIO transport (a locally-launched process, the common case for desktop agent integrations), or `spring-ai-starter-mcp-server-webflux` for a reactive HTTP server. All three auto-configure annotation scanning; none requires manually wiring `McpServerFeatures`.

```
# application.properties
spring.ai.mcp.server.protocol=STREAMABLE
spring.ai.mcp.server.type=SYNC
spring.ai.mcp.server.annotation-scanner.enabled=true
```

9.2 Add `@McpTool` alongside `@Tool`

`@McpTool` (`org.springframework.ai.mcp.annotation.McpTool`, from the `spring-ai-mcp-annotations` module pulled in by the starter above) is a separate annotation from `@Tool` because it targets a different consumer — an external MCP client rather than an in-process `ChatClient` — but it follows the identical naming and schema rules from §4–6. Add it to the same method:

```

package com.acme.orders.tools;

import org.springframework.ai.mcp.annotation.McpTool;
import org.springframework.ai.mcp.annotation.McpToolParam;
import org.springframework.ai.tool.annotation.Tool;
import org.springframework.stereotype.Component;

@Component
public class OrderTools {

    // ...constructor unchanged from §7.3...

    @Tool(
        name = "get_order_status",
        description = "Retrieves the current fulfillment status of one order. "" +
            "Call this when the customer asks about the status, tracking, "" +
            "or delivery date of an order they already placed. Requires "" +
            "the order ID; ask the customer for it if not yet known."
    )
    @McpTool(
        name = "get_order_status", // identical name – same tool, two front doors
        description = "Retrieves the current fulfillment status of one order. "" +
            "Call this when the customer asks about the status, tracking, "" +
            "or delivery date of an order they already placed. Requires "" +
            "the order ID; ask the customer for it if not yet known.",
        generateOutputSchema = true, // publish GetOrderStatusResponse's shape too
        annotations = @McpTool.McpAnnotations(
            title = "Get order status",
            readOnlyHint = true, // this tool never mutates state
            destructiveHint = false,
            idempotentHint = true // safe to retry with the same input
        )
    )
    public GetOrderStatusResponse getOrderStatus(GetOrderStatusRequest request) {
        // unchanged from §7.3
    }
}

```

THE ANNOTATION HINTS ARE NOT DECORATION

`readOnlyHint`, `destructiveHint`, and `idempotentHint` are read by MCP clients to decide whether a tool is safe to call speculatively or requires user confirmation first. A `cancel_order` tool must set `destructiveHint = true` and `readOnlyHint = false` — an external agent may otherwise call it without pausing for confirmation.

9.3 Verify external discovery

With the starter's default `STREAMABLE` protocol, start the application and list the registered tools against the MCP endpoint to confirm the schema an external agent will actually see:

```

curl -s http://localhost:8080/mcp \
  -H "Content-Type: application/json" \
  -d '{"jsonrpc":"2.0","id":1,"method":"tools/list"}' | jq .

```

HTTP MCP TRANSPORTS ARE UNAUTHENTICATED BY DEFAULT

`spring-ai-starter-mcp-server-webmvc` and `-webflux` expose the MCP JSON-RPC endpoint with no authentication out of the box. Before deploying anywhere reachable outside localhost, add a `TransportContextExtractor` to read and validate an `Authorization` header, and check it inside any tool whose `destructiveHint` is `true`. A tool with no side effects behind `readOnlyHint = true` is a lower-risk default to expose first.

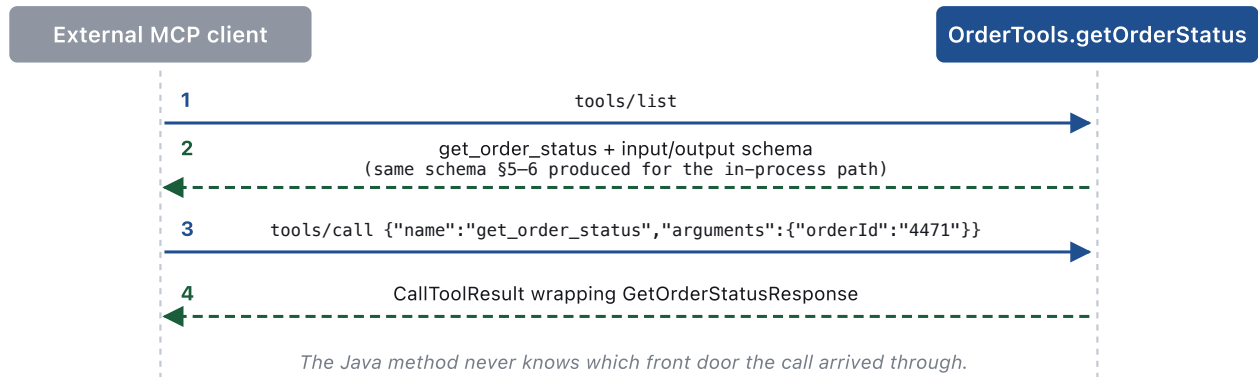


Figure 9.1 — External discovery and invocation. The MCP server layer translates the JSON-RPC envelope; `getOrderStatus` itself is the same method §7.3 wrote, called through the same request/response records.

10 Error Handling

A tool fails in exactly one of two shapes: an **expected** outcome the model should relay to the user (§7.3's "order not found"), or an **unexpected** failure the model should be told about so it can apologize and suggest a retry, not silently make something up.

FAILURE KIND	ENCODE AS	HANDLED BY
Expected: no result, empty result, business-rule rejection	A normal return value on the response record (a null field, an empty list, a status enum value)	Your method body, as ordinary data
Unexpected: I/O failure, timeout, invalid input that slipped past the schema	A thrown <code>RuntimeException</code> with a message written for the model to read	Spring AI's <code>ToolExecutionExceptionProcessor</code> , wrapping it in <code>ToolExecutionException</code>

By default, a `RuntimeException`'s message is sent back to the model as the tool result, and the model continues the conversation with that message as context — which is why the message in §7.4's

`IllegalStateException` reads as guidance ("ask the customer to try again shortly") rather than as a log line. Checked exceptions and `Error`s are re-thrown rather than swallowed.

```
# application.properties – invert the default so exceptions propagate
# instead of being converted to a model-visible message. Rarely what you want
# for a tool talking to an end user; occasionally correct for a tool used
# only in automated agent-to-agent pipelines with their own error handling.
spring.ai.tools.throw-exception-on-error=true
```

NEVER LET A RAW EXCEPTION MESSAGE REACH THE MODEL UNEDITED

`ex.getMessage()` from a JDBC driver or an HTTP client routinely contains connection strings, internal hostnames, or stack fragments. Catch the specific exception and re-throw with a message you wrote for this purpose, exactly as §7.4 does — never let a library's default exception message flow to the model unedited.

11 Testing Tools

A tool has two things worth testing independently: the schema Spring AI generates from your annotations, and the method's behavior when called with realistic model-supplied JSON. Both are ordinary JUnit tests — no model call, and no network, is required for either.

TESTING THE GENERATED SCHEMA

This is the automated form of the manual check in §7.6. Run it on every build so a missing `@ToolParam` or an accidentally-required optional field fails CI instead of surfacing as a model misbehaving in production.

```
@Test
void getOrderStatusSchema_marksIncludeHistoryOptional() {
    Method method = ReflectionUtils.findMethod(
        OrderTools.class, "getOrderStatus", GetOrderStatusRequest.class);

    ToolCallback callback = MethodToolCallback.builder()
        .toolDefinition(ToolDefinitions.builder(method).build())
        .toolMethod(method)
        .toolObject(new OrderTools(orderRepository))
        .build();

    var definition = callback.getToolDefinition();

    assertThat(definition.name().isEqualTo("get_order_status"));
    assertThat(definition.name().matches("^([a-z][a-z0-9]*(_[a-z0-9]+)*$")); // §4.1
    assertThat(definition.description().isNotBlank()); // §4.4

    DocumentContext schema = JsonPath.parse(definition.inputSchema());
    assertThat(schema.read("$.required", List.class)).containsExactly("orderId");
    assertThat(schema.read("$.properties.orderId.description", String.class)).isNotBlank();
}
```

TESTING TOOL BEHAVIOR DIRECTLY, WITHOUT A MODEL

Call `ToolCallback.call(String)` with the exact JSON shape a model would send. This exercises the real deserialization and serialization path — not just the Java method — while staying deterministic and fast.

```

@Test
void getOrderStatus_returnsNotFoundShape_whenOrderMissing() {
    when(orderRepository.findById("missing")).thenReturn(Optional.empty());

    String result = callback.call("""
        {"orderId": "missing"}
        """);

    DocumentContext json = JsonPath.parse(result);
    assertThat(json.read("$.status")).isNull();
    assertThat(json.read("$.orderId", String.class)).isEqualTo("missing");
}

@Test
void getOrderStatus_omitsHistory_whenIncludeHistoryFalse() {
    when(orderRepository.findById("4471")).thenReturn(Optional.of(SHIPPED_ORDER));

    String result = callback.call("""
        {"orderId": "4471", "includeHistory": false}
        """);

    DocumentContext json = JsonPath.parse(result);
    assertThat(json.read("$.history", List.class)).isEmpty();
    assertThat(json.read("$.status", String.class)).isEqualTo("SHIPPED");
}

```

TESTING THROUGH THE AGENT (INTEGRATION LEVEL)

Reserve a small number of end-to-end tests that run a real prompt through a real `ChatClient` against a live (or recorded) model, to confirm the model actually chooses this tool for representative prompts. This is the only layer of the three that costs real API calls and non-deterministic latency — keep it small, and let the two test types above carry the bulk of coverage.

```

@Test
void supportAgent_callsGetOrderStatus_forStatusQuestion() {
    String reply = supportAgent.prompt()
        .user("What's the status of order 4471?")
        .call()
        .content();

    verify(orderRepository).findById("4471");
    assertThat(reply).containsIgnoringCase("shipped");
}

```

12 Definition-of-Done Checklist

Copy this list into the pull request that adds or changes a tool. Every box must be checked before merge.

- ☐ Tool name is lowercase snake_case, matches `^[a-z][a-z0-9]*(_[a-z0-9]+)*$`, and is ≤ 64 characters (§4.1).
- ☐ Tool name follows the `verb_noun` or `domain_verb_noun` pattern, with a verb from the §4.2 table or an equally unambiguous synonym.
- ☐ `name` is set explicitly on `@Tool` — never left to default to the Java method name (§4.2).
- ☐ Tool name is unique across every tool set it can be registered into, not just within its own class (§4.1).
- ☐ Description states what the tool does, when to call it, and what input it needs — in that order (§4.4).
- ☐ Description does not restate the name and does not describe the implementation (§4.4).
- ☐ Input is a single dedicated `{ToolName}Request` record; no loose primitive parameters beyond a trailing `ToolContext` (§5).
- ☐ Every request field has a non-blank `@ToolParam(description = ...)` (§5).
- ☐ Optional fields are boxed types with `@ToolParam(required = false)`, not primitives, not `Optional<T>` (§5).
- ☐ Output is a single dedicated `{ToolName}Response` record; never a bare `String`, `Map`, or JPA entity (§6).
- ☐ Status-like fields use an enum, not a free-text `String` (§6).
- ☐ Nullable response fields document what null means (§6).
- ☐ Expected failure states (not found, empty, rejected) are modeled as return values, not exceptions (§7.3, §10).
- ☐ Thrown exception messages are written for the model, contain no internal detail (hostnames, stack fragments, SQL), and follow §10.
- ☐ Tool class is `@Component`-annotated and named `{Domain}Tools` in a `.tools` package (§4.3).
- ☐ A schema test asserts the tool name, description presence, and required-fields set (§11).
- ☐ At least one behavior test calls `ToolCallback.call(String)` with realistic JSON for each distinct return shape (success, empty/not-found, and any error path) (§11).
- ☐ If exposed over MCP: `@McpTool` is present with matching name/description, and `readOnlyHint` / `destructiveHint` / `idempotentHint` are set truthfully (§9.2).

- If the tool is side-effecting or destructive, this was a deliberate, reviewed decision — not an oversight of the §4.2 verb table.

13 Common Pitfalls and Anti-Patterns

ANTI-PATTERN	WHY IT FAILS, AND THE FIX
The god-tool: one method that takes an <code>action</code> string and branches internally (<code>"get"</code> , <code>"cancel"</code> , <code>"refund"</code>)	Collapses several distinct risk profiles — safe reads and destructive writes — into one schema the model cannot reason about separately, and defeats the MCP <code>destructiveHint</code> entirely, since one flag cannot describe three different actions. Split into one tool per verb, per §4.2.
Leaving <code>description</code> unset or trivial	The model has no way to decide when to call the tool and will either never call it or call it for the wrong prompts. Mandatory per §4.4.
Letting <code>@Tool</code> 's <code>name</code> default to the Java method name	A refactor that renames the method silently renames the tool, breaking any prompt engineering or evaluation suite tuned against the old name. Always set <code>name</code> explicitly.
Returning <code>Map<String, Object></code> "to stay flexible"	Produces a near-empty JSON Schema with no documented properties, pushes the parsing burden onto the model, and cannot generate a useful MCP <code>outputSchema</code> . Use a record (§6) every time; add a field later rather than starting untyped.
Two adjacent same-typed parameters instead of a request record	A model (or a refactor) can trivially swap <code>fromCurrency</code> / <code>toCurrency</code> with no compiler warning. §5's single-record rule eliminates the swap class of bug entirely.
Throwing the raw exception from a downstream client	Leaks internal detail to the model and, transitively, sometimes to the end user reading its response. Catch and re-throw with a model-appropriate message, per §10.
A destructive tool with no <code>destructiveHint</code> , or a false one	An external MCP client may call it speculatively or without confirmation, believing it safe. Set the MCP annotation hints truthfully, always — §9.2.
Business validation only in the JSON Schema (a regex in <code>@Pattern</code>) and nowhere else	A schema constrains what a well-behaved model sends; it does not stop a malformed or adversarial call from reaching the method, especially over an externally exposed MCP endpoint. Validate again inside the method body for anything security- or money-relevant.
Async or reactive tool signatures (<code>CompletableFuture<T></code> , <code>Mono<T></code>) on a synchronous <code>@Tool</code> method	Not supported by Spring AI's method-based tool reflection for synchronous registration; the tool silently fails to register or throws at startup. Keep <code>@Tool</code> methods synchronous; reactive return types are only valid on <code>@McpTool</code> methods registered with an async MCP server (§9).
Testing only through a live model call	Slow, non-deterministic, and expensive to run on every commit; a schema regression can hide behind a model that happens to compensate for it that day. Put the bulk of coverage in the schema and behavior tests from §11; keep model-in-the-loop tests few.

A Appendix: Annotation and API Reference

A.1 @Tool

org.springframework.ai.tool.annotation.Tool

ATTRIBUTE	DEFAULT	PURPOSE
name	method name	Tool identifier sent to the model. Always set explicitly — §4.2.
description	""	What/when/needs guidance for the model. Mandatory under this standard — §4.4.
returnDirect	false	When true, the result bypasses the model and returns straight to the caller. Only honored when every tool called in that round also has it true.
resultConverter	Jackson-based default	Custom ToolCallResultConverter implementation for non-standard serialization.

A.2 @ToolParam

org.springframework.ai.tool.annotation.ToolParam

ATTRIBUTE	DEFAULT	PURPOSE
description	""	Field-level guidance. Mandatory under this standard — §5.
required	true	Whether the model must supply this field. Use boxed types when false.

DESCRIPTION AND REQUIRED-FLAG PRECEDENCE (HIGHEST WINS)

1. @ToolParam (Spring AI)
 2. @JsonPropertyDescription / @JsonProperty(required=) (Jackson)
 3. @JsonClassDescription / @Schema(required=) (Jackson / Swagger)
 4. @Nullable (required only; Spring Framework / JSpecify)

Set @ToolParam explicitly regardless — do not rely on a lower-precedence fallback.

A.3 Core interfaces

TYPE	SHAPE
ToolDefinition org.springframework.ai.tool	name(), description(), inputSchema() — the wire contract for one tool.

TYPE	SHAPE
ToolCallback org.springframework.ai.tool	getToolDefinition() , getToolMetadata() , call(String) , call(String, ToolContext) .
ToolContext org.springframework.ai.tool	Carries data excluded from the model-visible schema (tenant id, user id). Populated by application code, not the model.
ToolCallbackProvider org.springframework.ai.tool	getToolCallbacks() — dynamic tool discovery at runtime.
MethodToolCallback org.springframework.ai.tool	Builder-based ToolCallback over a reflective method call. What @Tool produces under the hood.
FunctionToolCallback org.springframework.ai.tool	Builder-based ToolCallback over a Function / BiFunction / Supplier / Consumer reference — §8.
ToolExecutionException org.springframework.ai.tool	Wraps an exception thrown from a tool method before it reaches ToolExecutionExceptionProcessor — §10.

A.4 @McpTool / @McpToolParam

org.springframework.ai.mcp.annotation.McpTool / McpToolParam

ATTRIBUTE	DEFAULT	PURPOSE
name	method name	MCP tool identifier. Keep identical to the paired @Tool name — §9.2.
description	method name	Same what/when/needs standard as @Tool .
title	""	Human-facing UI label, distinct from the machine-facing name.
generateOutputSchema	false	Publish a JSON output schema derived from the return type. Set true for any non-primitive response record — §6, §9.2.
annotations	@McpAnnotations	Client hints: readOnlyHint , destructiveHint , idempotentHint , openWorldHint . Set truthfully — §9.2, §13.
metaProvider	DefaultMetaProvider	Supplies the MCP _meta field.

B Appendix: Complete Reference Implementation

The full, compilable listing this document builds incrementally in §7 and §9, collected here as a single reference. Package `com.acme.orders.tools`.

```
// -----  
// GetOrderStatusRequest.java  
// -----  
package com.acme.orders.tools;  
  
import org.springframework.ai.tool.annotation.ToolParam;  
  
public record GetOrderStatusRequest(  
    @ToolParam(description = "The order identifier, as shown on the customer's receipt or confirmation  
email.")  
    String orderId,  
  
    @ToolParam(  
        description = "Include the full status history, not just the current status.",  
        required = false  
    )  
    Boolean includeHistory  
) {}  
  
// -----  
// GetOrderStatusResponse.java  
// -----  
package com.acme.orders.tools;  
  
import java.time.Instant;  
import java.util.List;  
  
public record GetOrderStatusResponse(  
    String orderId,  
    OrderStatus status,  
    String trackingNumber,  
    List<StatusEvent> history  
) {  
    public enum OrderStatus { PENDING, PROCESSING, SHIPPED, DELIVERED, CANCELLED }  
    public record StatusEvent(OrderStatus status, Instant occurredAt) {}  
  
    public static GetOrderStatusResponse notFound(String orderId) {  
        return new GetOrderStatusResponse(orderId, null, null, List.of());  
    }  
}  
  
// -----  
// OrderTools.java  
// -----  
package com.acme.orders.tools;  
  
import com.acme.orders.tools.GetOrderStatusResponse.OrderStatus;  
import com.acme.orders.tools.GetOrderStatusResponse.StatusEvent;  
import java.util.List;
```

```

import org.springframework.ai.mcp.annotation.McpTool;
import org.springframework.ai.mcp.annotation.McpToolParam;
import org.springframework.ai.tool.annotation.Tool;
import org.springframework.stereotype.Component;

@Component
public class OrderTools {

    private final OrderRepository orderRepository;

    public OrderTools(OrderRepository orderRepository) {
        this.orderRepository = orderRepository;
    }

    @Tool(
        name = "get_order_status",
        description = "Retrieves the current fulfillment status of one order. """ +
            "Call this when the customer asks about the status, tracking, """ +
            "or delivery date of an order they already placed. Requires """ +
            "the order ID; ask the customer for it if not yet known."
    )
    @McpTool(
        name = "get_order_status",
        description = "Retrieves the current fulfillment status of one order. """ +
            "Call this when the customer asks about the status, tracking, """ +
            "or delivery date of an order they already placed. Requires """ +
            "the order ID; ask the customer for it if not yet known.",
        generateOutputSchema = true,
        annotations = @McpTool.McpAnnotations(
            title = "Get order status",
            readOnlyHint = true,
            destructiveHint = false,
            idempotentHint = true
        )
    )
    public GetOrderStatusResponse getOrderStatus(GetOrderStatusRequest request) {
        if (request.orderId() == null || request.orderId().isBlank()) {
            throw new IllegalArgumentException("An order ID is required to look up status.");
        }

        var order = orderRepository.findById(request.orderId()).orElse(null);
        if (order == null) {
            return GetOrderStatusResponse.notFound(request.orderId());
        }

        List<StatusEvent> history = Boolean.TRUE.equals(request.includeHistory())
            ? order.statusHistory().stream()
                .map(e -> new StatusEvent(OrderStatus.valueOf(e.status()), e.occurredAt()))
                .toList()
            : List.of();

        return new GetOrderStatusResponse(
            order.id(),
            OrderStatus.valueOf(order.status()),
            order.trackingNumber(),
            history
        );
    }

    // A second tool in the same class, demonstrating the destructive case §13 warns about.
    @Tool(
        name = "cancel_order",

```

```

        description = "Cancels an order that has not yet shipped. Call this only after "" +
            "the customer has explicitly confirmed they want to cancel; this "" +
            "action cannot be undone. Requires the order ID."
    )
    @McpTool(
        name = "cancel_order",
        description = "Cancels an order that has not yet shipped. Call this only after "" +
            "the customer has explicitly confirmed they want to cancel; this "" +
            "action cannot be undone. Requires the order ID.",
        generateOutputSchema = true,
        annotations = @McpTool.McpAnnotations(
            title = "Cancel order",
            readOnlyHint = false,
            destructiveHint = true,        // ← the flag that matters here
            idempotentHint = true         // cancelling twice leaves the same end state
        )
    )
    public CancelOrderResponse cancelOrder(CancelOrderRequest request) {
        var order = orderRepository.findById(request.orderId())
            .orElseThrow(() -> new IllegalArgumentException(
                "No order found with ID " + request.orderId() + "."));

        if (order.status().equals("SHIPPED") || order.status().equals("DELIVERED")) {
            return new CancelOrderResponse(request.orderId(), false,
                "Order has already shipped and can no longer be cancelled.");
        }

        orderRepository.cancel(request.orderId());
        return new CancelOrderResponse(request.orderId(), true, null);
    }

    public record CancelOrderRequest(
        @ToolParam(description = "The order identifier to cancel.")
        String orderId
    ) {}

    public record CancelOrderResponse(
        String orderId,
        boolean cancelled,
        // null when cancelled=true; explains why not, when cancelled=false
        String reason
    ) {}
}

```

C Appendix: Sources

This document was built from, and should be kept current against, the following primary sources.

- [Spring AI Reference — Tool Calling](#) — `@Tool`, `@ToolParam`, `ToolCallback`, `ToolDefinition`, `ToolContext`, JSON schema generation, exception handling, return-direct.
- [Spring AI Reference — MCP Server Boot Starter](#) — starter artifacts, transport configuration, annotation scanning.
- [Spring AI Reference — MCP Server Annotations](#) — `@McpTool`, `@McpToolParam`, output schema generation, client hints.
- [Spring AI Reference — Getting Started](#) — BOM coordinates, Spring Boot compatibility, starter naming.
- spring.io/projects/spring-ai — project overview, current release line, framing on multi-provider portability.
- habuma.com/springairecipes — Craig Walls' Spring AI Recipes, informing this document's testing and tool-observability framing (`ToolCallAdvisor`, tool-selection reasoning).

KEEP THIS APPENDIX CURRENT

Spring AI's tool-calling and MCP APIs have moved across recent minor versions — most notably the introduction of `@McpTool` as a dedicated MCP-facing annotation distinct from `@Tool`. Re-verify class and package names against the reference for the Spring AI version actually in use before treating §A as authoritative for a version other than 2.0.1.